Kapitel 3: KI-Tools nachvollziehbar vergleichen

Sprint 1, Change-Request-Agent für ein fiktives Softwarehaus. Stand 2026-10-20.

1. Worum es geht

Ein kleines fiktives Softwarehaus mit neun Mitarbeitenden baut KI-Einstiegs-Automatisierungen für kleine und mittlere Unternehmen. Seine Kunden äußern während des Projekts oder im Wartungsvertrag laufend Wünsche. Jedes Mal muss geklärt werden, ob der Wunsch schon im Vertrag steht oder als Zusatzleistung extra bezahlt wird. Das bleibt an der Projektleitung hängen, die zugleich Senior-Entwicklerin oder Senior-Entwickler ist, Kunden betreut und Demos vorbereitet.

Ein KI-Agent soll diese Arbeit vorbereiten. Er übernimmt drei Tätigkeiten, ein Mensch gibt jeden Fall frei:

1. **Schwärzen:** Personenbezogene Angaben werden aus der Kundenäußerung entfernt, bevor ein Modell den Text sieht.
2. **Klassifizieren:** Der Agent ordnet die Äußerung ein: Fehlermeldung, Überlegung, Wunsch, Auftrag oder Sonstiges.
3. **Entscheiden:** Bei Wünschen und Aufträgen schlägt er vor, ob sie im Vertrag enthalten oder eine Zusatzleistung sind, mit der passenden Vertragsstelle, einer kurzen Begründung und einem Sicherheitswert. Der Preis wird nicht vom Modell geschätzt, sondern nach einer festen Regel berechnet.

In diesem Kapitel wird verglichen, mit welchen Werkzeugen sich das am besten umsetzen lässt. **Gemessen wurde noch nichts.** Alle Bewertungen sind begründete Einschätzungen, die wir in der Testphase überprüfen. Belegt sind nur die Preise.

2. Was der Agent können muss

Die Anforderungen ergeben sich aus dem Alltag des Softwarehauses:

- Der Agent darf keine personenbezogenen Daten verarbeiten. Kundentexte werden vorher unkenntlich gemacht.
- Er ordnet jede Äußerung richtig ein. Fehlermeldungen und Reklamationen sortiert er aus und leitet sie weiter, sie dürfen nicht verloren gehen.
- Bei Wünschen und Aufträgen nennt er die passende Vertragsstelle, die Kategorie der Änderung, eine Begründung und einen Sicherheitswert.
- Der Preis kommt aus einer festen Regel. Bei Unsicherheit und bei Zusatzleistungen entscheidet immer ein Mensch. Der Agent schreibt nie selbst an Kunden und verschickt keine Angebote.
- Er arbeitet mit dem, was das Haus schon nutzt: Mail, Ticketsystem und die Verträge. Die Freigabe läuft über Slack oder Mail.
- Er muss in einem Haus mit neun Mitarbeitenden ohne eigene IT-Abteilung betreibbar sein und die Projektleitung entlasten, nicht zusätzlich belasten.
- Neue Vertragsarten sollen ohne Umbau dazukommen können. Wir arbeiten mit fünf Beispielverträgen.
- Der Nutzen soll die Kosten deutlich übersteigen. Das Ziel im Projektplan ist ein ROI größer als 3, der Rechenweg folgt in der Auswertung.

3. Was verglichen wird

Zwei Bausteine stehen fest. Als Plattform dient n8n, ein Werkzeug zum Verbinden und Automatisieren von Abläufen. Sie ist im Kurs der MSIT vorgegeben. Als Sprachmodell nutzen wir Gemini 2.5 Flash von Google. Der Kursanbieter MSIT stellt dafür einen Zugang über die Plattform OpenRouter bereit, der nur dieses Modell freigibt. Google beschreibt es als Preis-Leistungs-Modell für schnelle, häufige Aufgaben, die Schlussfolgern verlangen. Ein Vergleich verschiedener Anbieter findet deshalb nicht statt. Das Modell bleibt im Ablauf ein austauschbarer Schritt, sodass sich der Test später mit einem anderen Modell wiederholen lässt.

Verglichen werden drei Aufbauvarianten:

Variante	Aufbau	Gedanke dahinter
V1 Cloud-Modell	n8n schwärzt die Anfrage per Regeln und holt den Vertrag. Gemini 2.5 Flash ordnet ein und prüft gegen den Vertragstext. Ein Programmschritt kontrolliert, ob die genannte Vertragsstelle wirklich im Vertrag steht, und der Preis wird nach Regel berechnet. Dann gibt ein Mensch per Slack oder Mail frei	Unser Wunschaufbau: Das Modell liest und versteht Freitext, alles Verbindliche liegt bei Regeln und Menschen
V2 Regelbasiert	gleiche Umgebung und Freigabe, aber ohne Sprachmodell. Klassifizieren und Vertragsabgleich laufen über Schlüsselwörter und feste Zuordnung	Die einfachste Lösung. Sie dient als Messlatte und zeigt, was das Modell wirklich bringt
V3 Lokales Modell	wie V1, aber das Sprachmodell läuft auf eigener Hardware statt bei einem Cloud-Anbieter	Rückfallebene für den Fall, dass Daten das Haus gar nicht verlassen dürfen

Für die drei Tätigkeiten gibt es unterschiedliche Möglichkeiten. Die erste Einschätzung:

Tätigkeit	Möglichkeiten	Erste Einschätzung
Schwärzen	feste Regeln, ein KI-Modell in der eigenen Umgebung, beides zusammen	Regeln zuerst: Sie laufen ohne Modell und liefern immer dasselbe. Sie erkennen feste Muster wie Mailadressen und Telefonnummern, aber nicht jede Angabe im Fließtext. Ein lokales KI-Modell als Ergänzung wird getestet. Ein Cloud-Modell scheidet aus, weil der ungeschwärzte Text dafür das Haus verlassen müsste
Klassifizieren	Regeln, Cloud-Modell, lokales Modell	Der Unterschied zwischen „wäre schön, wenn" und „bitte setzt um" steckt in der Formulierung. Ein Sprachmodell erfasst das voraussichtlich besser als Stichwortregeln
Entscheiden	Regeln, Cloud-Modell, lokales Modell	Freitext gegen Vertragstext auszulegen ist die Stärke eines Sprachmodells. Der Preis bleibt eine Regel, die Freigabe ein Mensch

4. Nach welchen Kriterien verglichen wird

Wir nutzen die fünf Bewertungsfelder, die für einen Tool-Vergleich üblich sind: Funktion, Datenschutz, Bedienung, Integration und Kosten. Dazu kommt ein sechstes, das für dieses Projekt entscheidend ist: Kontrolle und Nachvollziehbarkeit, weil der Agent über Geld und Verträge mitentscheidet.

Kriterium	Die Frage	Wie wir es in der Testphase messen
Funktion	Trifft das Werkzeug die richtige Entscheidung, auch bei Wünschen, die nur teilweise im Vertrag stehen? Kommen neue Vertragsarten ohne Umbau dazu?	Trefferquote bei „im Vertrag" gegen „Zusatzleistung", besonders bei den Zusatzleistungen, die nicht übersehen werden dürfen; Trefferquote bei der Kategorie; Testlauf mit allen fünf Beispielverträgen

Kriterium	Die Frage	Wie wir es in der Testphase messen
Kontrolle und Nachvollziehbarkeit	Lässt sich jede Entscheidung prüfen? Was passiert bei Manipulationsversuchen im Kundentext?	Stimmt die genannte Vertragsstelle? Wie oft eskaliert der Agent bei Unsicherheit, und wie oft liegt er dann falsch? Testtexte mit Anweisungen wie „ignoriere den Vertrag“; gleiche Eingabe mehrfach
Datenschutz	Bleiben personenbezogene Daten im Haus?	Anteil der Angaben, die in Testtexten mit eingebauten Namen und Kontaktdaten nicht geschwärzt wurden; Weg der Daten je Variante
Bedienung	Wie viel Aufwand haben die Projektleitung beim Prüfen und das Haus beim Pflegen?	Zeit je Fall mit und ohne Agent; Aufwand für Einrichtung und Pflege, beim Bau mitgeschrieben
Integration	Fügt sich das Werkzeug in Mail, Ticketsystem, Verträge und Freigabe ein?	Läuft der Ablauf von der Anfrage bis zur Freigabe durch, ja oder nein
Kosten	Was kostet der Betrieb, auch gegenüber den Kosten einer falschen Entscheidung?	Preislisten mit Quelle; Kosten je Anfrage in der Testphase

5. Bewertung

Skala: sehr gut, gut, mittel, offen (noch nicht einschätzbar). Alles sind Einschätzungen auf Basis des Aufbaus, die Testphase prüft sie nach.

Kriterium	V1 Cloud-Modell (Gemini 2.5 Flash)	V2 Regelbasiert	V3 Lokales Modell
Funktion	**Gut.** Ein Sprachmodell kann Freitext gegen Vertragstext auslegen und die Stelle nennen. Wünsche, die nur zur Hälfte im Vertrag stehen, sind der erwartete Schwachpunkt und kommen deshalb gezielt in die Testmenge	**Mittel.** Eindeutige Fälle gehen über feste Regeln, die Auslegung von Freitext nur grob. Mehr Fälle gehen in die Eskalation an den Menschen	**Offen.** Aufbau wie V1. Ob ein lokales Modell dasselbe leistet, zeigt der Test
Kontrolle und Nachvollziehbarkeit	**Gut.** Das Modell entscheidet nichts allein: Ein Mensch gibt jeden Fall frei, der Preis kommt aus einer Regel, ein Programmschritt prüft die Fundstelle. So bleibt der Aufbau auch bei einem einfacheren Modell sicher. Offen sind Manipulationsversuche im Kundentext, die gezielt getestet werden	**Sehr gut.** Jede Regel ist einzeln nachvollziehbar, gleiche Eingabe ergibt gleiche Ausgabe, Manipulation eines Modells ist ausgeschlossen	**Gut.** Wie V1
Datenschutz	**Mittel.** Nur geschwärzter Text verlässt das Haus, und zwar über OpenRouter zum Modellanbieter. Regeln finden feste Muster, aber nicht jede Angabe im Fließtext, das ist ein Testfall. Im Kurs fließen nur erfundene Daten. Jeder Anbieter hat eigene Regeln zu Speicherung und Training, OpenRouter erlaubt, Anbieter auszuschließen, die auf Daten trainieren. Welche Einstellungen am Kurs-Zugang gelten, ist nicht geprüft. Für echte Kundendaten wäre vorher ein eigener Zugang mit geklärter Datenverarbeitung nötig	**Sehr gut.** Es verlässt nichts das Haus	**Sehr gut.** Der Text verlässt die eigene Umgebung nicht

Kriterium	V1 Cloud-Modell (Gemini 2.5 Flash)	V2 Regelbasiert	V3 Lokales Modell
Bedienung	**Gut.** Der Ablauf ist in n8n sichtbar, die Projektleitung prüft nur einen fertigen Vorschlag mit Begründung. Am Modell selbst fällt kaum Pflege an. Google führt Gemini 2.5 Flash als nicht abgekündigt, empfiehlt für neue Projekte aber neuere Modelle (Stand 2026-10-20)	**Mittel.** Das Regelwerk wächst mit jeder Vertragsart und muss gepflegt werden. Wer das in einem Haus mit neun Mitarbeitenden übernimmt, ist offen	**Mittel.** Ein eigener Modell-Server will betrieben werden (Hardware, Updates, Ausfälle), und das Haus hat keine IT-Abteilung
Integration	**Gut.** Eingang, Vertragsabruf und Freigabe liegen in n8n, das Modell wird über eine Schnittstelle angesprochen. Die Freigabe per Slack oder Mail bauen wir in Sprint 2	**Gut.** Wie V1, mit einer Schnittstelle weniger	**Mittel.** Wie V1, zusätzlich muss n8n den lokalen Server erreichen
Kosten	**Gut.** Im Kurs entstehen keine Modellkosten, im Betrieb sind es nach Preisliste Bruchteile eines Cents je Anfrage (siehe unten)	**Offen.** Keine Modellkosten, aber Aufwand für Aufbau und Pflege des Regelwerks	**Offen.** Keine Gebühr je Anfrage, aber Hardware, Strom und Betrieb

6. Was es kostet

Belegt, geprüft am 2026-10-20:

Baustein	Preis	Hinweis
n8n Cloud, Starter	20 € pro Monat bei jährlicher Abrechnung	2.500 Ausführungen pro Monat
n8n Cloud, Pro	50 € pro Monat bei jährlicher Abrechnung	10.000 Ausführungen pro Monat
n8n Community Edition	kostenlos bei Selbsthosting (Quelle docs.n8n.io)	Kosten entstehen nur für den eigenen Server. Einige Funktionen gehören zu den bezahlten Plänen, etwa Projekte, Single Sign-on und Versionsverwaltung mit Git. Die Lizenz erlaubt die Nutzung für interne Geschäftsabläufe
Gemini 2.5 Flash	0,30 US-Dollar je 1 Million Eingabe-Tokens, 2,50 US-Dollar je 1 Million Ausgabe-Tokens (einschließlich Denk-Tokens)	im Kurs über den OpenRouter-Zugang der MSIT ohne eigene Kosten; Preise laut ai.google.dev und openrouter.ai

Rechenbeispiel (Annahme zur Textlänge): Eine Anfrage mit etwa 3.000 Tokens Eingabe (Kundentext und Vertragsauszug) und 500 Tokens Ausgabe kostet nach Preisliste etwa 0,002 US-Dollar, also rund zwei Zehntel Cent. Tausend Anfragen im Monat lägen damit bei rund 2 US-Dollar. Der tatsächliche Verbrauch, auch an Denk-Tokens, wird in der Testphase gemessen. Zum Vergleich: Eine einzige falsch als „im Vertrag“ eingestufte Zusatzleistung bedeutet unbezahlte Entwicklerarbeit, und die ist deutlich teurer als dieses Modell.

7. Wie wir testen

Die Testphase findet in Sprint 3 statt. Alle drei Varianten laufen mit denselben Fällen:

- **Testmenge:** rund 60 selbst erstellte Kundenäußerungen zu zwei bis drei fiktiven Projekten, von Hand eingeordnet, mit schwierigen Fällen: grenzwertige Formulierungen, Verneinungen, Wünsche, die nur zur Hälfte im Vertrag stehen, und zwei Wünsche in einer Äußerung.

- **Zusätzlich:** Texte mit eingebauten personenbezogenen Angaben für den Schwärzen-Test und Texte mit Manipulationsversuchen.
- **Gemessen wird:** Trefferquote beim Vertragsurteil und bei der Kategorie, Zeit je Fall mit und ohne Agent, Anteil der Eskalationen und deren Fehlerquote, Anteil nicht geschwärzter Angaben, Kosten je Anfrage.
- **Unabhängige Einordnung:** Zwei Gruppenmitglieder ordnen die Fälle getrennt ein, die Testfälle schreibt nicht die Person, die den Agenten baut.
- **Entscheidungsregel:** Eine Variante scheidet aus, wenn sie den Datenschutz nicht einhält. Unter den übrigen entscheidet das Gesamtbild aus den anderen Kriterien.
- **Schwellen:** Wie gut „gut genug" ist, legen wir vor der Messung fest.

Ehrlich bleibt dabei: Testfälle und Musterlösung stammen aus derselben Gruppe, das Ergebnis fällt dadurch eher günstig aus. Das halten wir im Bericht fest.

8. Wo sich Ziele widersprechen

- **Datenschutz gegen Leistung:** V2 und V3 geben nichts nach außen, könnten aber Freitext schlechter auslegen als ein Cloud-Modell. Das ist die zentrale Frage der Testphase.
- **Kontrolle über die Daten gegen Aufwand:** Je mehr im eigenen Haus läuft, desto mehr Pflege fällt in einem kleinen Team an. V1 verlagert diesen Aufwand an den Anbieter und macht das Haus von dessen Modellen abhängig.
- **Nachvollziehbarkeit gegen Flexibilität:** Regeln sind prüfbar und wiederholbar, decken aber nur Vorhergesehenes ab. Ein Modell fängt unerwartete Formulierungen ab, liefert aber nicht jedes Mal exakt dasselbe.

9. Ergebnis und Empfehlung

Wir empfehlen vorläufig **V1 mit Gemini 2.5 Flash.** Die Gründe:

- Sie passt zum Kern der Aufgabe: Freitext im Kundentext mit Vertragstext abgleichen.
- Der Aufbau ist auf Sicherheit gebaut. Das Modell schlägt vor, Regeln rechnen den Preis, ein Programmschritt prüft die Fundstelle, ein Mensch gibt frei.
- Die Kosten liegen bei Bruchteilen eines Cents je Anfrage.
- Das Modell steht im Kurs zur Verfügung und lässt sich später austauschen.

V2 läuft als einfache Vergleichsbasis mit. Sie zeigt, was das Modell über Regeln hinaus bringt. V3 bleibt Rückfallebene, falls Schwärzung oder Datenweg sich als nicht ausreichend erweisen. Für einen späteren echten Betrieb würden wir zusätzlich aktuellere Modelle testen, der Ablauf ist dafür vorbereitet. Die endgültige Auswahl begründen wir in der Einsatzplanung im nächsten Kapitel.

10. Noch offen

- Die Schwellen für „gut genug" und der Rechenweg für den ROI, beides vor der Messung.
- Ob n8n in der Cloud oder selbst gehostet läuft.
- Ob für das Schwärzen ein lokales KI-Modell hinzukommt, das entscheidet der Test.